

Sistemas Cognitivos com LLMs [26E2_3]

ANVISA-LLM: Um Pipeline RAG para Consulta de Dados da ANVISA

Aluna: Renata Braga de Abreu

Dependências

Antes de iniciar o projeto neste *notebook*, instalar as dependências contidas no *requirements.txt*.

```
$ pip install -r ./requirements.txt
```

Descomente uma linha abaixo conforme o seu ambiente:

```
# ## OSX
# !pip install torch==2.11.0 torchvision==0.26.0 torchaudio==2.11.0
#
# ## WINDOWS / LINUX
#
# ## AMD RADEON
# # ROCM 7.2 (Linux only)
# !pip install torch==2.11.0 torchvision==0.26.0 torchaudio==2.11.0 --index-url
https://download.pytorch.org/whl/rocm7.2
#
# # Nvidia
# # CUDA 12.6
# !pip install torch==2.11.0 torchvision==0.26.0 torchaudio==2.11.0 --index-url
https://download.pytorch.org/whl/cu126
# # CUDA 12.8
# !pip install torch==2.11.0 torchvision==0.26.0 torchaudio==2.11.0 --index-url
https://download.pytorch.org/whl/cu128
# CUDA 13.0
# !pip install torch==2.11.0 torchvision==0.26.0 torchaudio==2.11.0 --index-url
https://download.pytorch.org/whl/cu130
# # CPU only
# !pip install torch==2.11.0 torchvision==0.26.0 torchaudio==2.11.0 --index-url
https://download.pytorch.org/whl/cpu
```

```
import torch
device = torch.device("cuda" if torch.cuda.is_available() else
                      ("xpu" if torch.xpu.is_available() else "cpu"))

device

device(type='cuda')
```

```
import numpy as np
import pandas as pd
import sys
import sklearn
import faiss

import sentence_transformers
import torch
from pathlib import Path

print(f"Python: {sys.version.split()[0]}")

print(f"NumPy: {np.__version__}")
print(f"Pandas: {pd.__version__}")
print(f"Scikit-Learn: {sklearn.__version__}")

print(f"PyTorch: {torch.__version__}")
if device.type == 'cuda':
    print(f'CUDA version: {torch.version.cuda}')
elif device.type == 'xpu':
    print(f'XPU version: {torch.version.xpu}')
print(f"FAISS: {faiss.__version__}")
print(f"SentenceTransformers: {sentence_transformers.__version__}")
```

```
Python: 3.12.12
NumPy: 2.4.4
Pandas: 2.3.3
Scikit-Learn: 1.9.0
PyTorch: 2.11.0+cu130
CUDA version: 13.0
FAISS: 1.14.3
SentenceTransformers: 5.6.0
```

Parâmetros

```
K_VEC_SIZE = 35
CHUNK_SIZE = 512
CHUNK_OVERLAP = 50
BATCH_SIZE = 32
MAX_NEW_TOKENS = 1024
DO_SAMPLE = True
TEMPERATURE = 0.2
```

Importação dos *datasets*

```

DATASET_DIR = 'dataset'
DATASET_FILENAMES = ['Anvisa_Alertras_2026-01-01_2026-06-19.csv',
'Anvisa_REGISTRO_MEDICAMENTOS.csv']
LOG_DIR = 'logs'
ROOT_DIR = Path().resolve().__str__()
DATASET_PATH = ROOT_DIR + '/' + DATASET_DIR
DATASET_FILEPATHS = [Path(DATASET_DIR + '/' + file + '/') for file in DATASET_FILENAMES]
Path.mkdir(Path(DATASET_PATH), parents=True, exist_ok=True)
LOG_PATH = Path(ROOT_DIR + '/' + LOG_DIR)
Path.mkdir(LOG_PATH, exist_ok=True)

```

Importa os arquivos de *dataset*:

```
df_collection = [pd.read_csv(DATASET_FILEPATH) for DATASET_FILEPATH in DATASET_FILEPATHS]
```

```
df_collection[0]
```

	titulo	data	url
0	Alerta 5265 (Tecnovigilância) - Comunicado da ...	16/06/2026	http://antigo.anvisa.gov.br/informacoes-tecnic...
1	Alerta 5264 (Tecnovigilância) - Comunicado da ...	16/06/2026	http://antigo.anvisa.gov.br/informacoes-tecnic...
2	Alerta 5263 (Tecnovigilância) - Comunicado da ...	16/06/2026	http://antigo.anvisa.gov.br/informacoes-tecnic...
3	Alerta 5262 (Tecnovigilância) - Comunicado da ...	16/06/2026	http://antigo.anvisa.gov.br/informacoes-tecnic...
4	Alerta 5261 (Tecnovigilância) - Comunicado da ...	16/06/2026	http://antigo.anvisa.gov.br/informacoes-tecnic...
...	...	...	...
3514	Alerta 1789 Atualizado (Tecnovigilância) - GE ...	13/01/2016	http://antigo.anvisa.gov.br/informacoes-tecnic...
3515	Alerta 1788	12/01/2016	http://antigo.anvisa.gov.br/informacoes-tecnic...
3516	Alerta 1787	12/01/2016	http://antigo.anvisa.gov.br/informacoes-tecnic...
3517	Alerta 1780	06/01/2016	http://antigo.anvisa.gov.br/informacoes-tecnic...
3518	Alerta 1779	06/01/2016	http://antigo.anvisa.gov.br/informacoes-tecnic...

3519 rows × 4 columns

df_collection[1]

	codigo	medicamento	situacao	data_Vencimento_Registro	categoria_Regulatoria_De
0	1098085	sinvastatina	Ativo	2036-04-01	G
1	466454	AMATO	Ativo	2026-11-01	
2	56508	cyfenol	Ativo	2034-11-01	BAIXO
3	1376475	letrozol	Ativo	2032-01-01	G
4	520950	ACETILCISTEÍNA	Ativo	2027-09-01	G
...	...	...	...	...	
43298	2760	PATANOL	Inativo	2018-06-01	
43299	11529	CLOREXIDINA 4% / CHLOROHEX	Inativo	NaN	BAIXO
43300	1230991	HEMOFIL	Inativo	2029-05-01	
43301	813586	NEO GENTAMICIN	Inativo	2026-07-01	
43302	17796	N?O DECLARADO	Inativo	NaN	DINAMI

43303 rows × 10 columns

Estas informações foram extraídas do site da ANVISA utilizando o método Web Scrapping. Veja [anexo 1](#) e [anexo 2](#) para maiores detalhes deste processo.

Embedding: *dividir para conquistar*

Este trecho de código realiza o **pré-processamento e a vetorização de dados** para o sistema de busca. O modelo *BAAI/bge-m3* carrega, formata e divide textos de duas bases de dados [Anvisa_Alertas_2026-01-01_2026-06-19.csv](#), [Anvisa_REGISTRO_MEDICAMENTOS.csv](#) em pedaços menores (*chunks*).

```
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer("BAAI/bge-m3", device=str(device))

Loading weights:   0%|          | 0/391 [00:00<?, ?it/s]
```

```
def chunk_text(docs, chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP):
    from langchain_text_splitters import RecursiveCharacterTextSplitter
    splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size,
    chunk_overlap=chunk_overlap)
    chunks = splitter.split_text("\n\n".join(docs))
    return chunks
```

```
docs = []
for i, row in df_collection[1].iterrows():
    doc = \
        f"""
Medicamento: {row['medicamento']}
Situação atual: {row['situacao']}
Data de Vencimento do Registro: {row['data_Vencimento_Registro']}
Categoria Regulatória: {row['categoria_Regulatoria_Descricao']}
Tipo autorização: {row['tipo_Autorizacao']}
Empresa: {row['razao_Social']}
CNPJ: {row['cnpj_Formatado']}
Número do processo: {row['numero_Processo_Formatado']}
Fonte: {row['url']}
=====
"""
    docs.append(doc)
```

```
chunked = chunk_text(docs)
docs.clear()
```

```
for i, row in df_collection[0].iterrows():
    doc = \
        f"""
{df_collection[0]['content']}

Fonte: {df_collection[0]['url']}
=====
"""
    docs.append(doc)
```

```
new_chunked = chunk_text(docs)
chunked.extend(new_chunked)
```

```
del new_chunked, docs, df_collection
torch.cuda.empty_cache()
```

Após liberar memória da GPU, o código transforma os *chunks* em vetores numéricos (*embeddings*), deixando-os prontos para serem indexados e consultados pelo sistema de recuperação.

```
embeddings = embedder.encode(chunked, chunk_size=CHUNK_SIZE, batch_size=BATCH_SIZE,
                              show_progress_bar=True,
                              convert_to_numpy=True)
```

```
Batches:   0%|          | 0/1794 [00:00<?, ?it/s]
```

FAISS: *em busca do contexto perfeito*

A função `get_context` implementa a etapa de busca de um sistema RAG (*Retrieval-Augmented Generation*).

Ela pega a pergunta (query), a transforma em um vetor numérico e usa a biblioteca FAISS para encontrar os trechos de texto mais parecidos dentro do *embedding* (onde estão as informações compiladas). Para otimizar o uso de memória, os cálculos são feitos em baixa precisão (`float16`). Por fim, ela junta os trechos encontrados em um único texto, que será usado como contexto para o modelo LLM formular a resposta.

```
def get_context(_query: str, _k=K_VEC_SIZE):
    import faiss
    import numpy as np
    dimension = embeddings.shape[1]
    index = faiss.IndexFlatIP(dimension)
    index.add(np.array(embeddings, dtype=np.float16))
    query_embedding = embedder.encode(
        [_query],
        normalize_embeddings=True
    )

    scores, ids = index.search(
        np.array(query_embedding, dtype=np.float16),
        k=_k
    )

    context_chunks = [chunked[i] for i in ids[0]]

    context = "\n\n".join(context_chunks)

    return context
```

LLM: *ligando os pontos*

A escolha do modelo *Ministral-3-3B-Instruct-2512* (uma versão quantizada do *Ministral-3-3B-Instruct-2512-BF16*) se deve à sua capacidade multilingual, de lidar com informações com uma grande quantidade de input tokens, de ser executado em hardwares com menores capacidade (8GB VRAM) e não depender de uma arquitetura robusta.

Foram realizada outras tentativas com outros modelos, tais como o *Ministral-3-3B-Instruct-2512-BF16* (versão base do modelo anterior), porém o desempenho foi inferior se comparando à versão que utiliza variáveis FP8.

Também foi realizada uma tentativa com o modelo *meta-llama/Llama-3.2-1B-Instruct*, porém havia uma limitação de quantidade máxima de *tokens* do contexto.

```
X_t_Y_k = pd.read_excel(ROOT_DIR + '/logs/compilado.xlsx')

import seaborn as sns

fig = sns.lineplot(data=X_t_Y_k, x='K_exec', y='seconds', hue='model')
fig.set_xlabel('K (chunks / query)')
fig.set_ylabel('Tempo (segundos)')
```

O gráfico acima compara o tempo de processamento (em segundos) de dois modelos em função do número de chunks por query (K):

- O tempo de execução aumenta de forma não linear à medida que mais *chunks* são adicionados.
- A versão BF16 (laranja) mostrou-se mais lenta que a versão FP8 (azul), o que foi uma surpresa.
- Pico entre K=25 e K=30, mas cai de forma inesperada quando K=35, em ambos modelos.
- O uso de `float16` na etapa FAISS pode ter influenciado a pouca diferença de desempenho entre os modelos, por já estarem otimizados.

Processar mais *chunks* torna o sistema significativamente mais lento, e a versão BF16 não trouxe ganhos de desempenho neste cenário específico.

```
# from transformers import Mistral3ForConditionalGeneration, AutoTokenizer
#
# model_id = "mistralai/Ministral-3-3B-Instruct-2512-BF16"
#
# # mistralai/Ministral-3-3B-Instruct-2512-BF16
# model = Mistral3ForConditionalGeneration.from_pretrained(model_id, device_map=device)
#
# tokenizer = AutoTokenizer.from_pretrained(model_id,
#                                           use_fast=True,
#                                           trust_remote_code=True)
```

```
from transformers import Mistral3ForConditionalGeneration, AutoTokenizer
from transformers.utils.quantization_config import FineGrainedFP8Config

# !pip install -U mistral-common kernels
torch.cuda.empty_cache()
torch.clear_autocast_cache()
model_id = "mistralai/Ministral-3-3B-Instruct-2512"

model = Mistral3ForConditionalGeneration.from_pretrained(model_id,
                                                         device_map=device,
                                                         quantization_config=FineGrainedFP8Config(dequantize=True))
```

Define o *tokenizer* para o modelo selecionado.

```
# Define o tokenizer
# Rodar sempre que redefinir um modelo

tokenizer = AutoTokenizer.from_pretrained(
    model_id,
    use_fast=True,
    trust_remote_code=True
)
```

Define as instruções que serão passadas à LLM, para orientar e limitar a sua resposta, evitando possíveis incoerências ou alucinações.

```
task = [
    "Você é um assistente que responde apenas com base no contexto.",
    "Priorize a extração de data, motivo e número do alerta ou qualquer outro número ou código que embase a citação / nome da empresa e CNPJ.",
    "Priorize alertas mais recentes.",
    "Sempre exiba o CNPJ quando citar uma empresa. Se o CNPJ não estiver no contexto, não o invente.",
    "Sempre mencione o número do alerta/processo/etc quando descrever ou realizar alguma afirmação.",
    "Sempre mencione a fonte (url).",
    "Use a formatação dd/mm/aaaa para datas.",
    "Se a resposta não estiver no contexto, declare que não sabe. Não invente informação. Não altera o link da fonte.",
    "Use hyperlink para cada URL.",
    "Não cite o contexto."
]
```

Monta a mensagem que será enviada para a LLM, com as instruções acima, o contexto (informações extraídas) e a pergunta.

```
def build_message(task=None, context=None, query=None):
    return \
        [
            {
                "role": "system",
                "content": " | ".join(map(str, task)) if task is not None else "",
            },
            {
                "role": "user",
                "content": f"Contexto: {context}",
            },
            {
                "role": "user",
                "content": f"Pergunta: {query}",
            }
        ]
```


Chama o `build_message` e usa como entrada no `tokenizer` fornecido pelo modelo para aplicar o *template*.

```
def get_input_model(_query: str, _context: str):
    messages = build_message(task, _context, _query)
    return tokenizer.apply_chat_template(
        messages,
        return_tensors="pt",
        add_generation_prompt=True,
        skip_special_tokens=True,
        # truncation=True,
    ).to(model.device)
```

Chama o modelo LLM. O parâmetro `args` é um `dict` que agrupa todos os inputs em um só.

```
def call_model_generate(args: dict):
    from time import time, gmtime
    torch.cuda.empty_cache()
    ini = time()
    outputs = (
        model.generate(
            **args['input_ids'],
            max_new_tokens=args['max_new_tokens'],
            max_length=args['max_length'],
            do_sample=args['do_sample'],
            temperature=args['temperature'],
        ))
    fim = time()
    exe_tim = gmtime(fim - ini)
    return outputs, exe_tim
```

Pergunta (*query*)

```
query = "Quais as medicações mais citadas?"
```

```
K_vec_size_input = 15.0
```

Execução

Monta o contexto com base na *query* # Monta a mensagem e codifica # Monta a entrada da LLM # Chama a LLM # Decodifica a saída da LLM (vetor para tokens) # Exibe a resposta gerada / decodificada

```
def ask_llm(query: str, k_vec_size=K_VEC_SIZE):
    # Monta o contexto com base na query
    context = get_context(query, k_vec_size)

    # Monta a mensagem e codifica
    inputs = get_input_model(query, context)

    # Monta entrada do modelo da LLM
    args = dict({
        'input_ids': inputs,
        'max_new_tokens': MAX_NEW_TOKENS,
        'max_length': None,
        'do_sample': DO_SAMPLE,
        'temperature': TEMPERATURE,
    })

    # Chama a LLM
    outputs, exec_time = call_model_generate(args)

    # Decodifica a saída e corta para exibir somente a resposta
    answer = tokenizer.decode(outputs[0],
                              skip_special_tokens=True).split(query)[1]

    return answer, exec_time
```

```
# Redefine o tamanho do vetor conforme o _input_
K_VEC_SIZE = int(K_vec_size_input) if K_VEC_SIZE > 0 else K_VEC_SIZE

answer, exec_time = ask_llm(query, K_VEC_SIZE)

# Exibe em formato Markdown
from IPython.display import Markdown

Markdown(answer)

<IPython.core.display.Markdown object>
```

Grava *log* da execução

```
def export_log(k_vec_size, exec_time):
    import time
    with open(LOG_PATH.absolute().__str__() + '\\\\' + model_id.split('/')[1] + '_' + time.
strftime('%Y%m%d%H%M%S', time.localtime()) + '_k=' + str(k_vec_size) + '.txt', 'w',
encoding='utf-8') as l:
        print('*****',
file=l)
        print('Parâmetros:', file=l)
        print(f'model_id: {model_id}', file=l)
        print(f'k_vector_size = {k_vec_size}', file=l)
        print(f'batch_size = {BATCH_SIZE}', file=l)
        print(f'chunk_size = {CHUNK_SIZE}', file=l)
        print(f'chunk_overlay = {CHUNK_OVERLAP}', file=l)
        print(f'do_sample = {DO_SAMPLE}', file=l)
        print(f'temperature = {TEMPERATURE}', file=l)
        print(f'Execution time = {time.strftime('%Hh %Mm %Ss', exec_time)}', file=l)
        print(f'Device = {device.type}', file=l)
        print('*****',
file=l)
        print(f'Pergunta: \n{query}', end='\n\n', file=l)
        print(f'Resposta:\n{answer}', file=l)
        l.close()
```

```
export_log(k_vec_size=K_VEC_SIZE, exec_time=exec_time)
```

Benchmark

O código abaixo foi usado para gerar *logs* (que podem ser verificados no diretório [logs](#)) para realizar comparações entre os modelos, utilizando o mesmo contexto, pergunta, tamanho do *chunk* e *overlap*, variando somente o *K*, que é a quantidade de *chunks* que o modelo irá coletar antes de começar a gerar as respostas e o próprio modelo.

Os resultados desses testes foram compilados para o arquivo [compilado.xlsx](#), que é importado da seção [LLM](#) para ilustrar e realizar a comparação entre os modelos.

```
# K_exec = [5, 10, 15, 20, 25, 30, 35]
# X_t_Y_k = pd.DataFrame(data={})
```

```
# Tempo_exec = []
#
# for k in K_exec:
#     answer, exec_time = ask_llm(query, k)
#     Tempo_exec.append((int(time.strftime('%M', exec_time)) * 60) + int(time.strftime('%S', exec_time)))
#     export_log(k, exec_time)
#
# X_t_Y_k.insert(len(X_t_Y_k.columns) , model_id.split('/')[1], Tempo_exec )
```